

4 STRUKTURNO PROGRAMIRANJE i potprogramiranje

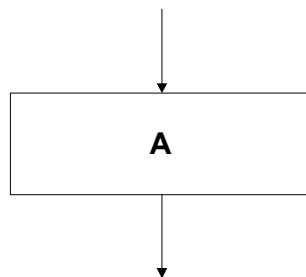
- 4.1 *Pravilni programi*
- 4.2 *Strukturni teorem*
- 4.3 *Strukturirani programi*
- 4.4 *Razvitak programa u koracima preciziranja*
- 4.5 *Sortiranje: studijski primjer analize performansi programskih rješenja*
- 4.6 *Potprogrami - opća svojstva*
- 4.7 *Nezavisne funkcije i njihovi formalni, lokalni, globalni i stvarni argumenti*
- 4.8 *Interne funkcije i procedure*
- 4.9 *Koprogrami, kaskadna i protočna obrada*

4 STRUKTURNO PROGRAMIRANJE I POTPROGRAMIRANJE

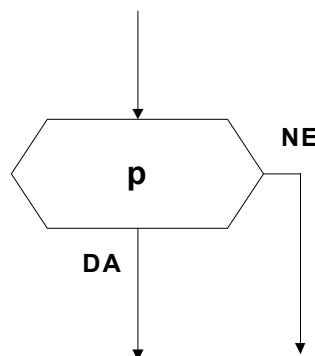
4.1 Pravilni programi

Svaki dijagram toka predstavlja usmjereni graf čiji čvorovi mogu biti (1) **proces**, (2) **predikat** i (3) **kolektor**. Njihove definicije su slijedeće:

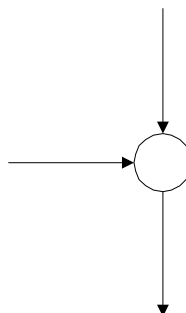
DEFINICIJA 1: Element dijagrama toka sa jednom ulaznom i jednom izlaznom linijom u kome se obavlja **obrada ili prijenos** podataka naziva se **PROCES** (alternativni nazivi su funkcijski čvor ili obrada, a najčešća realizacija procesa je u vidu instrukcije dodjele vrijednosti).



DEFINICIJA 2: PREDIKAT je element dijagrama toka sa jednim ulazom i dva izlaza koji **obavlja** isključivo **kontrolnu funkciju**: izračunava vrijednost pridruženog logičkog izraza i zatim usmjerava daljnji tok obrade u pravcu jedne od izlaznih linija ako je izračunata vrijednost true, a u pravcu druge od izlaznih linija ako je izračunata vrijednost false.



DEFINICIJA 3: KOLEKTOR je element dijagrama toka sa dvije ulazne linije i jednom izlaznom linijom koji ne obavlja nikakvu obradu i koji se primjenjuje onda kada dva nezavisna toka obrade usmjeriti u nekom zajedničkom pravcu.



Strukturirano programiranje razlikuje strukturirane i nestrukturirane programe, a za strukturirane programe temeljan je zahtjev da program bude pravilan.

DEFINICIJA 4: Pod pojmom PRAVILAN PROGRAM (engl. proper program) podrazumijeva se program čiji dijagram toka zadovoljava slijedeća tri kriterija:

- (1) Program ima samo jednu ulaznu liniju,
- (2) Program ima samo jednu izlaznu liniju,
- (3) Za svaki čvor (proces, kolektor ili predikat) postoji putanja koja ide od ulazne linije kroz čvor do izlazne linije.

Posljednji od navedenih kriterija ukazuje da pravilni programi ne smiju imati beskonačne petlje ili izolirane (nedostižne) programske segmente.

DEFINICIJA 5: Svaki sastavni dio pravilnog programa (tj. neki podgraf u dijagramu toka) koji je također pravilan program naziva se PRAVILAN POTPROGRAM.

Prema ovoj definiciji zaključujemo da svi procesi u sastavu pravilnog programa imaju status pravilnog potprograma, dok predikati nisu pravilni potprogrami zato jer imaju po dva izlaza. Naravno, ni kolektori nisu pravilni potprogrami jer imaju po dva ili više ulaza.

DEFINICIJA 6: Pravilan program čiji svaki sastavni dio koji je pravilan potprogram sadrži samo jedan čvor, naziva se JEDNOSTAVAN PROGRAM.

Jednostavni programi se ne mogu dalje dekomponirati u pravilne potprograme, pa najviše primjera jednostavnih programa nalazimo među osnovnim kontrolnim strukturama.

4.2 Strukturni teorem

Strukturni teorem se pojavio 1966. godine u članku "Flow diagrams, Turing machines and languages with only two formation rules", čiji su autori Corrado Böhm i Giuseppe Jacopini. Oni su u svom radu promatrali slijedeće tri osnovne kontrolne strukture:

SEKVENCA

SELEKCIJA

ITERACIJA

i stavili su sebi u zadatak ustanoviti pod kojim uvjetima se programi mogu realizirati superpozicijom navedenih osnovnih kontrolnih struktura. Za pravilne programe to je, kao što ćemo vidjeti, uvijek moguće.

DEFINICIJA 7: Skup jednostavnih programa čijom superpozicijom se može realizirati proizvoljan pravilan program nazivamo BAZA STRUKTURIRANIH PROGRAMA. Baza strukturiranih programa može biti dovoljna, ako se izostavljanjem neke od kontrolnih struktura iz baze dobiva neka jednostavnija baza. U suprotnom baza je potrebna.

DEFINICIJA 8: FORMALNO STRUKTURIRANI PROGRAM je svaki pravilan program sačinjen korištenjem jedino kontrolnih struktura iz određene baze.

DEFINICIJA 9: Dva programa su EKVIVALENTNA onda kada realiziraju istu obradu podataka, tj. kada za iste ulazne podatke generiraju iste rezultate.

Za svaki pravilan program može se formirati funkcija veze. Najprije se svi procesi i predikati numeriraju tako da se okolina datog programa tretira kao "nulti čvor", a zatim se svi čvorovi u dijagramu toka proizvoljno označe nizom prirodnih brojeva od 1 do n. Sa 1 se označava početni

čvor, a to je onaj čvor na koji je kod pravilnog programa vezana jedina postojeća ulazna linija. Za ostale čvorove redoslijed numeriranja čvorova nije bitan.

Kada su čvorovi numerirani može se uvesti logička funkcija veze

$L : [0...n]^2 \rightarrow [F..T]$

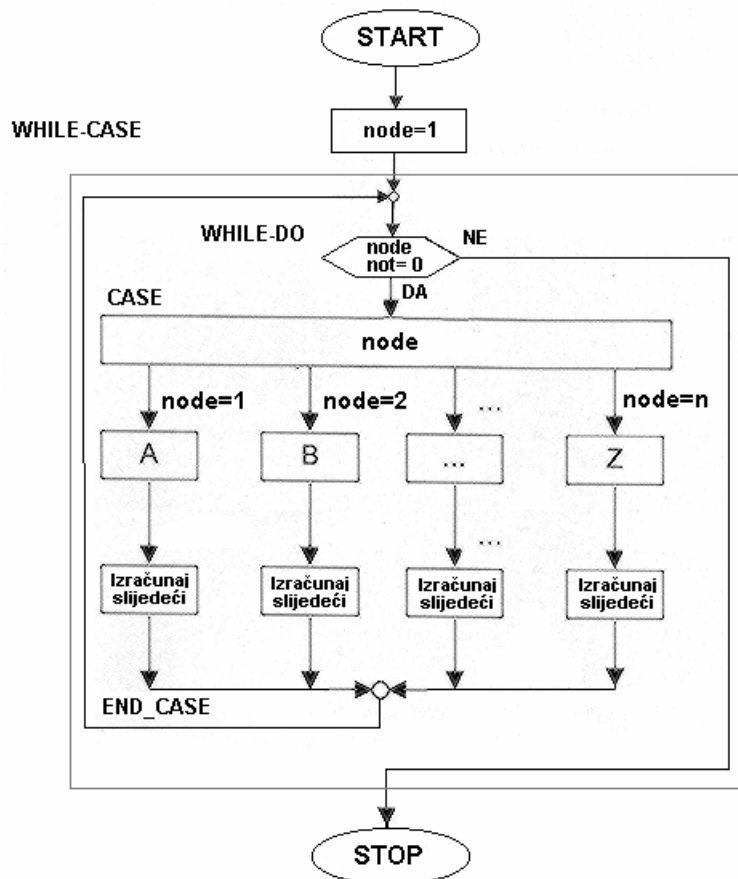
čije vrijednosti definiramo na slijedeći način:

DEFINICIJA 10: Vrijednost $L(i,j)$ funkcije veze određuje vezu od čvora i do čvora j u dijagramu toka. Ako je $L(i,j)=T$ onda veza postoji, a ako je $L(i,j)=F$ onda veze nema. Za svaku vrijednost i postoji samo jedan j takav da $L(i,j)=T$.

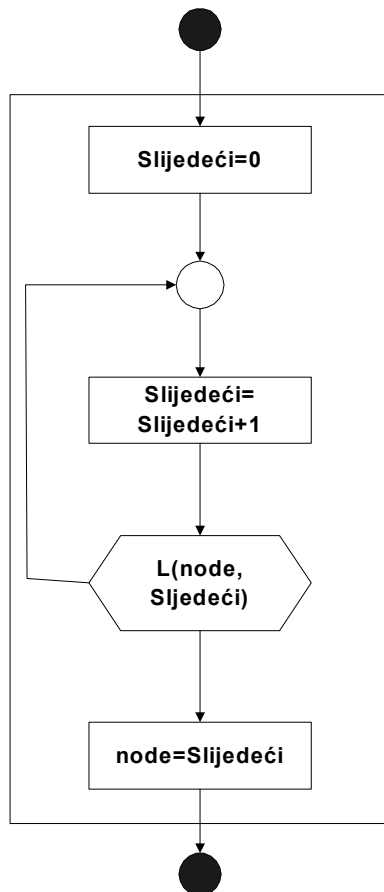
Vrijednosti funkcije $L(i,j)$ mogu biti logičke konstante ili u slučaju predikata, logički izrazi.

Najjednostavnije je da se funkcija veze definira u tabličnoj formi.

TEOREM 1: (Teorem o kanoničnoj formi) Za svaki pravilan program postoji ekvivalentan program koji se sastoji od REPEAT-UNTIL (ili WHILE-DO) petlje unutar koje se nalazi CASE struktura sa onoliko paralelnih grana koliko ima čvorova u dijagramu toka. REPEAT-CASE ili WHILE-CASE forma programa naziva se kanonična forma programa.



Postupak izračunavanja slijedećeg čvora možemo vidjeti na slijedećem dijagramu:



TEOREM 2: (Osnovni strukturni teorem) Svaki pravilan program može se transformirati u ekvivalentan formalno strukturiran program uz korištenje baze od tri osnovne kontrolne strukture (sekvenca, selekcija i iteracija) primjenjene na procese i predikate polaznog programa, i uz moguću primjenu jedne dopunske selektorske varijable.

```

DEFINE node : CARDINAL ;
node := 1 ;
WHILE node <> 0 DO
  IF node=1
  THEN
    Obrada za čvor 1 ;
    node := redni broj odlaznog čvora
  ELSE IF node=2
  THEN
    Obrada za čvor 2 ;
    node := redni broj odlaznog čvora
  
```

```

.....
        ELSE IF node=n
            THEN
                Obrada za čvor n ;
                node := redni broj odlaznog čvora
            END_IF
        .....
    END_IF
END_IF
END_WHILE

```

Budući da je ovaj program u potpunosti ekvivalentan kanoničnoj WHILE-CASE formi i da se sastoji isključivo od struktura sekvence, selekcije i iteracije, on je formalno strukturiran i ekvivalentan proizvoljnom programu.

TEOREM 3: (Strukturni teorem sa dvije kontrolne strukture) Svaki pravilan program može se transformirati u ekvivalentan formalno strukturiran program uz korištenje baze od samo dvije osnovne kontrolne strukture (sekvenca i iteracija) primjenjene na procese i predikate polaznog programa, uz moguću primjenu dopunskih selektorskih i logičkih varijabli.

DOKAZ: Da bi dokazali ovaj teorem možemo se u potpunosti osloniti na dokaz osnovnog strukturnog teorema, pod uvjetom da možemo pokazati da se selekcija može realizirati pomoću sekvence, iteracije i dopunskih varijabli. U tom cilju promatrajmo selekciju

IF p THEN A ELSE B END_IF

i slijedeći program koji koristi pomoćne logičke varijable "alfa" i "beta":

```

alfa := p ;
beta := not(p) ;

WHILE alfa DO
    A ;
    alfa := false
END_WHILE ;

WHILE beta DO
    B ;
    beta := false
END_WHILE

```

4.3 Strukturirani programi

Krajnja posljedica strukturnog teorema, koju je opazio i istakao i Dijkstra, je da za konstrukciju proizvoljnih dijagrama toka nije potrebno posjedovati GO TO naredbu jer kontrolne strukture obuhvaćene strukturnim teoremom predstavljaju dovoljan skup. Stoga se GO TO naredba, kao jedna od uzorka nestrukturiranih programa može u principu izbaciti iz svih viših programskih jezika. Dijkstra je to u eksplicitnoj formi i zatražio tvrdeći da je kvaliteta programera inverzno proporcionalna gustoći GO TO naredbi u njihovim programima i da je GO TO naredba suviše primitivna, te djeluje kao izazov za proizvodnju nestrukturiranih i nečitkih programa.

GO TO naredba je ipak preživjela sve "napade" i održala se u višim programskim jezicima. Razlog je jednostavan: nije kriva GO TO naredba, već je kriv način na koji se ona koristi.

4.4 Razvitak programa u koracima preciziranja

Po definiciji strukturirano programiranje je razvitak programa u koracima preciziranja čime se dobivaju programi u formi hijerarhijskih struktura programskih iskaza i objekata koji se programom obrađuju. Ovom tehnikom se postižu slijedeći ciljevi:

- minimizacija broja grešaka tokom procesa razvoja programske opreme,
- minimizacija rada potrebnog za razvitak korektnih programa i
- minimizacija troškova održavanja programske opreme.

U teoriji strukturiranog programiranja definira se pravilan strukturirani program kao pravilan program koji je dobiven postupkom razvitka programa u koracima preciziranja uz primjenu ograničenog broja osnovnih kontrolnih struktura. Stoga programe možemo klasificirati po stupnju strukturiranosti na slijedeći način:

- nepravilni programi,
- pravilni nestrukturirani programi,
- pravilni formalno strukturirani programi i
- pravilni strukturirani programi.

Razvitak programa u koracima preciziranja je jednostavna i prirodna tehnika koju su najviše popularizirali E. Dijkstra (u knjizi "Structured Programming", Academic Press, 1972) i N. Wirth (u članku "Program Development by Stepwise Refinement", CACM, April 1971). Sam postupak se može opisati na slijedeći način:

- (1) Formulirati problem u formi pogodnoj za računalno rješavanje.
- (2) Formulirati osnovnu ideju algoritamskog rješenja.
- (3) Napisati osnovne komponente programskog rješenja u vidu niza komentara.
- (4) Izdvojiti pogodnu manju cjelinu (iskazanu u vidu komentara) i rastaviti je na detaljnije programske zahtjeve.
- (5) Ponavljati korak (4) dok se ne dobiju programski zahtjevi koji su toliko jednostavni da se mogu realizirati u vidu programskih segmenata na pseudojeziku.
- (6) Odabrati neki od programskih zahtjeva i realizirati ga na pseudojeziku koristeći pri tome jedino kontrolne strukture iz određene usvojene baze. Na početku programa dati definicije potrebnih podataka.
- (7) Sustavno ponavljati korak (6) dok god je to moguće i pri tome povećavati stupanj detalja kako za programe tako i za podatke koje program obrađuje.

Na kraju opisanog postupka dobiva se program formuliran na pseudojeziku. Takav program je u dovoljnoj mjeri blizak realnim programskim jezicima da se može bez teškoća direktno prevesti na neki od realnih programskih jezika.

Razvitak programa u koracima preciziranja - primjer: testiranje da li je zadani broj prosti broj

```
DEFINE      N, K, delta, counter : cardinal
            prime : logical

write 'Input : N = '; read N ;
while N>1 do

if N<4 then prime:=true else
    if (N mod 2) = 0 then prime:=false else
        if (N mod 3) = 0 then prime:=false else
            if N<25 then prime:=true else
                K := int(sqrt(N)) ;
                delta := 4 ;
                counter := 5 ;
                repeat
                    prime := (N mod counter) > 0 ;
                    delta := 6 - delta ;
                    counter := counter + delta
                until not(prime) or counter>K
            end_repeat
        end_if
    end_if
end_if ;
if prime then write N, ' is prime' else write N, ' is not prime' end_if;

write 'Input : N = '; read N
end_while
```

(rješenje u pseudokodu)

P A S C A L

Program Primetest (input, output) ;

Var N, K, delta, counter : integer ;
 prime : Boolean ;

BEGIN

```
    write('Input : N = ') ; read(N) ;
    while N>1 do
    begin
        if N<4 then prime := true else
            if (N mod 2) = 0 then prime := false else
                if (N mod 3) = 0 then prime := false else
                    if N<25
                    then prime := true
                    else
                        begin
                            K := trunc(sqrt(N)) ;
                            delta := 4 ;
                            counter := 5 ;
                            repeat
                                prime := (N mod counter > 0 ;
                                delta := 6 - delta ;
                                counter := counter + delta
                            until (not prime) or (counter>K)
                        end;
                    if prime
                    then writeln(N:7, ' is prime')
                    else writeln(N:7, ' is NOT prime') ;
                    write('Input : N = ') ; read (N)
        end
    end
```

END.

F O R T R A N

```
      INTEGER N, K, delta, counter
      LOGICAL prime

10    write(*,*) 'Input N'
      read(*,*) N
        if (N .LT. 2) STOP
        if (N .LT. 4) then
          prime = .true.
        else if (mod(N,2) .EQ. 0) then
          prime = .false.
        else if (mod(N,3) .EQ. 0) then
          prime = .false.
        else if (N .LT. 25) then
          prime = .true.
        else
          K = sqrt(real(N))
          delta = 4
          counter = 5
20          prime = mod(N, counter) .GT. 0
            delta = 6 - delta
            counter = counter + delta
            if (prime .AND. counter .LE. K) GO TO 20
        end if
        if (prime) then
          write(*,*) N, ' is prime'
        else
          write(*,*) N, ' is NOT prime'
        end if
      go to 10
END
```

BASIC

```
10 DEF FNMOD(k,j) = k - int(k/j) * j
11   INPUT ' Input: N= '; N
15   if N<2 then STOP
20   if N<4 then prime=1: goto 75
25   if FNMOD(N,2)=0 then prime=0: goto 75
30   if FNMOD(N,3)=0 then prime=0: goto 75
35   if N<25 then prime=1: goto 75
40   K = int(sqrt(N))
45   delta = 4
50   counter = 5
55       if FNMOD(N,counter)>0 then prime=1 else prime=0
60       delta = 6 - delta
65       counter = counter + delta
70       if prime=1 AND counter<=K then goto 55
75   if prime=1 then print N; ' is prime' else print N; ' is NOT prime'
80 goto 11
```

Primjer rezultata:

Input: N=9
9 is NOT prime

Input: N=1009
1009 is prime

Input: N=1
Stop

KORAK PRECIZIRANJA #1.

```
{Učitati broj N};  
{Ispitati da li je broj N prosti};  
{Prikazati poruku o rezultatu testiranja}
```

KORAK PRECIZIRANJA #2.

```
{Učitati N ; ako je  $N < 2$  ponoviti učitavanje};  
{Ako  $N < 4$  onda  $\text{prime} := \text{true}$ , preskačemo ostale testove i prikazujemo poruku};  
{Odrediti granicu testiranja  $K := \lceil \sqrt{N} \rceil$ };  
{Ako je N nije djeljiv sa 2,3,...,K onda  $\text{prime} := \text{true}$ , inače  $\text{prime} := \text{false}$ };  
{Ako je  $\text{prime} = \text{true}$  prikazati poruku da je N prosti broj, a u protivnom prikazati poruku da N nije prosti broj}
```

KORAK PRECIZIRANJA #3.

```
DEFINE      N, K : cardinal ;  
            prime : logical ;  
REPEAT read N UNTIL  $N > 1$  END_REPEAT ;  
{Ako  $N < 4$  onda  $\text{prime} := \text{true}$ , preskačemo ostale testove i prikazujemo poruku}  
IF  $N < 4$  THEN  $\text{prime} := \text{true}$   
ELSE  
    {Ako je N paran onda  $\text{prime} := \text{false}$  i preskačemo ostale testove};  
    {Ako je N djeljiv sa 3 onda  $\text{prime} := \text{false}$  i preskačemo ostale testove};  
    {Ako  $N < 25$  onda  $\text{prime} := \text{true}$  i preskačemo ostale testove};  
     $K := \lceil \sqrt{N} \rceil$ ;  
    {Ako  $N \geq 25$  onda testirati djeljivost sa 5, 7, 11, 13, 17, ..., K i postaviti vrijednost  
semafora prime};  
END_IF;  
IF prime THEN write N, 'is prime' ELSE write N, 'is not prime' END_IF
```

KORAK PRECIZIRANJA #4.

```
DEFINE      N, K : cardinal ;  
            prime : logical ;  
REPEAT read N UNTIL  $N > 1$  END_REPEAT;  
  
{Ako  $N < 4$  onda  $\text{prime} := \text{true}$ , preskačemo ostale testove i prikazujemo poruku};  
IF  $N < 4$  THEN  $\text{prime} := \text{true}$  ELSE  
    {Ako je N paran onda  $\text{prime} := \text{false}$  i preskačemo ostale testove};  
    IF  $(N \bmod 2) = 0$  THEN  $\text{prime} := \text{false}$  ELSE  
        {Ako je N djeljiv sa 3 onda  $\text{prime} := \text{false}$  i preskačemo ostale testove};  
        IF  $(N \bmod 3) = 0$  THEN  $\text{prime} := \text{false}$  ELSE  
            {Ako  $N < 25$  onda  $\text{prime} := \text{true}$  i preskačemo ostale testove};  
            IF  $N < 25$  THEN  $\text{prime} := \text{true}$  ELSE  
                 $K := \lceil \sqrt{N} \rceil$ ;  
  
                {Ako  $N \geq 25$  onda testirati djeljivost sa 5, 7, 11, 13, 17, ..., K i  
postaviti vrijednost semafora prime}  
            END_IF  
        END_IF  
    END_IF  
END_IF
```

```

        END_IF
    END_IF
END_IF;

```

```

IF prime THEN write N, 'is prime' ELSE write N, 'is not prime' END_IF

```

KORAK PRECIZIRANJA #5.

```

DEFINE      N, K, delta, counter : cardinal ;
            prime : logical ;
REPEAT read N UNTIL N>1 END_REPEAT;

```

```

{Ako N<4 onda prime:=true, preskačemo ostale testove i prikazujemo poruku};
IF N<4 THEN prime:=true ELSE

```

```

    {Ako je N paran onda prime:=false i preskačemo ostale testove};
    IF (N mod 2) = 0 THEN prime:=false ELSE

```

```

        {Ako je N djeljiv sa 3 onda prime:=false i preskačemo ostale testove};
        IF (N mod 3) = 0 THEN prime:=false ELSE

```

```

            {Ako N<25 onda prime:=true i preskačemo ostale testove};
            IF N<25 THEN prime:=true ELSE

```

```

                K := int(sqrt(N));

```

```

                {Ako N>=25 onda testirati djeljivost sa 5, 7, 11, 13, 17, ..., K i
postaviti vrijednost semafora prime}

```

```

                    delta :=4;
                    counter :=5;

```

```

                    REPEAT
                        prime := (N mod counter) > 0;
                        delta := 6 - delta;          {delta :=2, 4, 2, 4, ..., itd.}
                        counter := counter+delta      {counter :=7, 11, 13, 17, ...,
itd.}
                    UNTIL not(prime) OR counter>K
                    END_REPEAT

```

```

                END_IF
            END_IF
        END_IF

```

```

END_IF

```

```

IF prime THEN write N, 'is prime' ELSE write N, 'is not prime' END_IF

```

4.5 Sortiranje: studijski primjer analize performansi programskih rješenja

Sortiranje je praktičan problem koji se javlja u obradi podataka bilo samostalno, bilo kao sastavni dio raznih složenijih problema. Algoritmi sortiranja predstavljaju primjer razlika u efikasnosti raznih algoritamskih rješenja istog problema, a također pokazuju kako algoritamska rješenja mogu evoluirati uz stalno poboljšavanje i usavršavanje.

U problematici sortiranja obično se razlikuje interno sortiranje i sortiranje na eksternim memorijama.

Pod internim sortiranjem podrazumijeva se situacija gdje se u memoriji računala nalazi vektor $X(1), X(2), \dots, X(N)$ čije su vrijednosti slučajni brojevi. Kod rastućeg sortiranja potrebno je sadržaj vektora X preurediti tako da bude ispunjen uvjet $X(1) \geq X(2) \geq \dots \geq X(N)$. Između rastućeg i padajućeg sortiranja nema nikakvih bitnih algoritamskih razlika.

Kod sortiranja na eksternim memorijama radi se po pravilu o relativno velikim datotekama čiji zapisi sadrže polje ključa i polja podataka, a problem se sastoji u tome da se zapisi datoteke poredaju po rastućim (ili padajućim) vrijednostima ključa. Da bi se to postiglo vektori ključeva se unose u operativnu memoriju i tu interno sortiraju pa se na temelju rezultata internog sortiranja pristupa preuređivanju sadržaja datoteke. Stoga interno sortiranje predstavlja središnji problem u području sortiranja.

Klasične metode internog sortiranja obuhvaćaju slijedeće:

1. Metoda umetanja (Insertion sort)
2. Metoda zamjene susjeda (Bubble sort)
3. Metoda izbora (Selection sort)
4. Sortiranje po lancima ekvidistantnih zapisa (Shell sort)
5. Sortiranje po metodi kupa (Heapsort)
6. Particijska metoda sortiranja (Quicksort).

Cilj je pokazati osnovne tehnike programiranja u BASIC-u, kao i neke opće karakteristike programskih rješenja. BASIC program koji slijedi u nastavku je studijski primjer koji pokazuje slijedeće:

- način realizacije programa za komparaciju efikasnosti algoritma,
- način mjerenja vremena primjenom BASIC funkcije TIMER,
- primjenu BASIC funkcije SWAP,
- tehniku prikaza rezultata u vidu tablica čiji se oblik i veličina automatski podešavaju u funkciji od zadanih parametara,
- parametarsko dimenzioniranje vektora podataka,
- primjenu generatora pseudoslučajnih brojeva u komparaciji algoritama,
- opseg razlika performansi između efikasnih i neefikasnih algoritama.

```

10 REM -----
20 REM          KOMPARACIJA OSNOVNIH ALGORITAMA SORTIRANJA
30 REM -----
40 REM Parametri:
50 REM  NSORTS =      broj analiziranih algoritama sortiranja
60 REM  QUANT =       dimenzija najkraćeg vektora koji se sortira
70 REM                  (dužine vektora koji se sortiraju su i*QUANT,
80 REM                  i=1, ..., M ; M je također jedan od parametara)
90 REM  VEC =         referentni vektor slučajnih brojeva za sortiranje
100 REM -----
110 REM
120 REM  Inicijalizacija parametara:
130 REM
140      NSORTS = 8 ; QUANT = 50 ; M = 6
150      DIM VEC(M*QUANT), X(M*QUANT), SORT$(NSORTS), SL(QUANT), SR(QUANT)
160      FOR I=1 TO M*QUANT ; VEC(I) = 10000*RND ; NEXT I
170 REM
180 INPUT "BASIC language procesor = "; B$
190 REM
200 SORT$(1) = "INSERTION SORT"
210 SORT$(2) = "UNIDIRECTIONAL BUBBLE SORT"
220 SORT$(3) = "BIDIRECTIONAL BUBBLE SORT"
230 SORT$(4) = "SELECTION SORT"
240 SORT$(5) = "SELECTION SORT (REDUCED SWAP)"
250 SORT$(6) = "SHELL SORT"
260 SORT$(7) = "HEAPSORT"
270 SORT$(8) = "QUICKSORT"
280 REM
290 PRINT : PRINT "COMPARISON OF SORT ALGORITHMS"
300 IF LEN(B$)>0 THEN PRINT "BASIC compiler/interpreter = "; B$
310 PRINT
320 PRINT TAB(35); "R U N   T I M E S   [SEC]"
330 PRINT STRING$(30+8*M, "-")
340 PRINT TAB(16); "VECTOR SIZE: ";
350 FOR I=1 TO M : PRINT USING "#####"; I*QUANT; : NEXT I:PRINT
360 PRINT STRING$(30+8*M, "-")
370 REM
380 REM ----- Mjerenje -----
400 FOR K=1 TO NSORTS
410     PRINT SORT$(K) ; TAB(30);
420     FOR N=QUANT TO M*QUANT STEP QUANT
430         REM
440         REM ----- Priprema vektora X -----
450         REM
460         FOR I=1 TO N : X(I)=VEC(I) : NEXT i
470         REM
480         REM ----- Mjerenje brzine sortiranja -----
490         REM
500         T1 = TIMER
510         ON K GOSUB 600, 670, 770, 870, 950, 1040, 1160, 1300
520         T2 = TIMER
530         PRINT USING "#####.##"; T2-T1;
540     NEXT N : PRINT
550 NEXT K : PRINT STRING$(30+8*M, "-") : END

```

560

P O T P R O G R A M I

```

570 REM -----
580 REM          INSERTION SORT (METODA UMETANJA)
590 REM -----
600 FOR I=2 TO N : T=X(I) : J=I-1
610     IF X(J)>T THEN X(J+1)=X(J) : J=J-1 : IF J>0 THEN 610
620     X(J+1) = T
630 NEXT I : RETURN

```

```

640 REM -----
650 REM  UNIDIRECTIONAL BUBBLE SORT (JEDNOSMJERNA ZAMJENA SUSJEDA)
660 REM -----
670 L=N-1
680 F=0
690 FOR I=1 TO L
700     IF X(I)>X(I+1) THEN SWAP X(I), X(I+1) : F=1
710 NEXT I
720 L=L-1 : IF L*F<>0 THEN 680
730 RETURN

```

```

740 REM -----
750 REM  BIDIRECTIONAL BUBBLE SORT (DVOSMJERNA ZAMJENA SUSJEDA)
760 REM -----
770 L=N-1 : D=1 : I=0
780 F=0
790 FOR J=1 TO L : I=I+D
800     IF X(I)>X(I+1) THEN SWAP X(I), X(I+1) : F=1
810 NEXT J : D=-D
820 L=L-1 : IF L*F<>0 THEN 780
830 RETURN

```

```

840 REM -----
850 REM          SELECTION SORT (METODA IZBORA)
860 REM -----
870 FOR I=1 TO N-1
880     FOR J=I+1 TO N
890         IF X(I) > X(J) THEN SWAP X(I), X(J)
900     NEXT J
910 NEXT I : RETURN

```

```

920 REM -----
930 REM  SELECTION SORT WITH REDUCED SWAP (UBRZANA METODA IZBORA)
940 REM -----
950 FOR I=1 TO N-1 : L=I
960     FOR J=I+1 TO N
970         IF X(L) > X(J) THEN L=J
980     NEXT J
990     IF L>I THEN SWAP X(L), X(I)
1000 NEXT I : RETURN

```



```

1010 REM -----
1020 REM   SHELL SORT (SORTIRANJE PO LANCIMA EKVIDISTANTNIH ZAPISA)
1030 REM -----
1040 G=N
1050 IF G<=1 THEN RETURN
1060 G=INT(G/2) : L=N-G
1070 F=0
1080 FOR I=1 TO L
1090     H=I+G : IF X(I) > X(H) THEN SWAP X(I), X(H) : F=1
1100 NEXT I
1110 IF F=1 THEN 1070
1120 GOTO 1050

```

```

1130 REM -----
1140 REM                               HEAPSORT (KUP SORT)
1150 REM -----
1160 U=N : FOR L=INT(N/2) TO 1 STEP -1 : GOSUB 1190 : NEXT L
1170 L=1 : FOR U=N-1 TO 1 STEP -1:SWAP X(1), X(U+1):GOSUB 1190:NEXT U
1180 RETURN
1190 I=L : T=X(I)
1200     C=2*I
1210     IF C>U THEN 1260
1220     IF C<U THEN IF X(C+1)>X(C) THEN C=C+1
1230     IF T>=X(C) : THEN 1260
1240     X(I)=X(C) : I=C
1250     GOTO 1200
1260 X(I)=T : RETURN

```

```

1270 REM -----
1280 REM                               QUICKSORT (PARTICIJSKA METODA SORTIRANJA)
1290 REM -----
1300 I1=1 : J1=N : P=0
1310 I=I1 : J=J1: F=-1
1320 IF X(I)>X(J) THEN SWAP X(I), X(J) : F=-F
1330 IF F=1 THEN I=I+1 ELSE J=J-1
1340 IF I<J THEN 1320
1350 IF I+1<J1 THEN P=P+1 : SL(P)=I+1 : SR(P)=J1
1360 J1=I-1 : IF I1<J1 THEN 1310
1370 IF P>0 THEN I1=SL(P) : J1=SR(P) : P=P-1 : GOTO 1310
1380 RETURN
1390 REM -----

```

Prikazani program sastoji se od zaglavlja (linije 10-100), pripremnog dijela (linije 110-370), mjernog dijela (linije 380-550) i potprograma za sortiranja (linije 560-1390). U pripremnom dijelu definiraju se parametri, dimenzioniraju se vektori i formira se referentni vektor slučajnih brojeva VEC koji će se zatim sortirati pomoću svih prikazanih algoritama. Pri tome se vektor VEC organizira kao M dijelova dužine QUANT. M je promjenljiva veličina pa se u linijama 330-360 ispisuje zaglavlje tablice promjenljive dužine. Sadržaj tablice ispisuje se u dijelovima pomoću linija 410, 530-550. Da bi vremena sortiranja bila uspoređljiva potrebno je da svi algoritmi sortiraju isti vektor. Stoga se u liniji 460 prenosi sadržaj vektora VEC u vektor X i tako inicijalizirani vektor X se sortira. Vrijeme sortiranja mjeri se tako da se trenutno vrijeme u računalu očita neposredno prije i poslije sortiranja (linije 500 i 520). Zatim se izračuna odgovarajuća vremenska razlika.

Prvi prikazani algoritam (metoda umetanja, linije 600-630) može se realizirati pomoću najkraćeg programa koji je moguće zapisati koristeći samo tri linije BASIC-a. Za prikazivanje koncepcije ovog algoritma pretpostavimo da je I-1 komponenti vektora X već sortirano:

$$X(1) \leq X(2) \leq \dots \leq X(I-1).$$

I-tu komponentu X(I) treba umetnuti na odgovarajuće mjesto u vektor X da bi se dobio sortirani niz od I komponenti. Pri tome se komponente koje su veće od X(I) moraju pomaknuti za jedno mjesto u desno na slijedeći način (u uvjetima kada je $X(J) > X(I)$ odnos među elementima):

$$x(1) \quad X(2) \dots X(J-1) \quad *** \quad X(J) \quad X(J+1) \dots X(I-1) \quad X(I)$$

Najmanji broj pomaka je 0 (ako je $X(I-1) \leq X(I)$), a najveći broj pomaka je I-1 (uslučaju da je $X(I) < X(1)$). Ako bi raspodjele vrijednosti brojeva bile takve da se u prosjeku uvijek pomakne pola skupine sortiranih brojeva onda bi srednji broj pomaka za umetanje I-tog broja iznosio 0.5 (I-1). U tom slučaju ukupan broj pomaka za I=2, 3, ..., N bi u prosjeku iznosio:

$$P = 0.5 (1 + 2 + \dots + N-1) = 0.25 N (N-1) \\ \approx 0.25 N^2 \quad (N \gg 1) .$$

Budući da je vrijeme rada programa proporcionalno proju pomaka slijedi da se radi o kvadratnom algoritmu tj. da je vremenska složenost ovog algoritma $O[N^2]$. Ako pretpostavimo da je vrijeme rada računala opisano s $T = cN^2$ onda se konstanta c može odrediti iz zadnje točke u gornjoj tablici, što daje vrijednost koeficijent $c=0.0036$.

Koristeći funkciju $T(N)=0.0036N^2$ za $N=50, \dots, 300$ dobivamo:

$$T(50) = 9 \text{ sec}, T(100) = 36 \text{ sec}, T(150) = 81 \text{ sec}, T(200) = 144 \text{ sec}, T(300) = 324 \text{ sec},$$

što se podudara s izmjerenim rezultatima.

Može se s razlogom postaviti pitanje je li korektno strukturiran program:

```
600   FOR I=2 TO N : T=X(I) : J=I-1
610       IF X(J)>T THEN X(J+1)=X(J) : J=J-1 : IF J>0 THEN 610
620   X(J+1)=T : NEXT I
```

U liniji 610 nalazi se petlja s dva izlaza koja ne predstavlja ni jednu od temeljnih kontrolnih struktura. Ako se strogo krećemo u okviru temeljnih kontrolnih struktura, onda bi jedan korektan zapis navedenog algoritma bio:

for i=2 to n do

```

t := x [ i ] ;
j := i-1 ;
while j>0 and x [ j ]>t do
    x [ j+1 ] := x [ j ] ;
    j := j-1
end_while;
x [ j+1 ] := t
end_for

```

Druga varijanta navedenog algoritma ima za cilj pojednostavniti (i ubrzati) uvjet koji se ispituje u WHILE petlji. Da bi uvjet $x [j] > t$ bio dovoljan, neophodno je uvesti dopunski element $x [0] = t$ koji ima ulogu graničnika, odnosno svojom vrijednošću osigurava izlaz iz petlje kada se pomakne (ako je to potrebno) i prvi element vektora:

```

for i=2 to n do
    t := x [ i ] ;
    x [ 0 ] := t ;    {umetanje graničnika}
    j := i-1 ;
    while x [ j ] > t do
        x [ j+1 ] := x [ j ] ;
        j := j-1
    end_while ;
    x [ j+1 ] := t
end_for

```

BASIC program mogao bi se napisati i prema bilo kojem od navedenih dobro strukturiranih algoritama. Prikazani BASIC program svojom konciznošću odražava duh i stil programiranja u BASIC-u. S druge strane dovoljno je jednostavan da se može lako razumjeti način rada koji se njime ostvaruje. Ako bi se u ime konciznosti, strukturiranost i razumljivost nekog BASIC programa u primjetnoj mjeri narušila, onda treba ustrajati na strogoj primjeni načela strukturiranog programiranja.

4.6 Potprogrami - opća svojstva

Osnovna ideja inženjerskog pristupa problemima: složeni problem sustavno dekomponirati u relativno jednostavne potprobleme, svaki problem nezavisno riješiti i sva dobivena pojedinačna rješenja agregirati sa ciljem dobivanja globalnog rješenja polaznog složenog problema. Ova se filozofija redovito primjenjuje pri realizaciji složenih programskih sustava. Pri tome se mehanizam razlaganja problema na potprobleme svodi na razlaganje složenog programa na potprograme.

Primjer:

$$R(x, y, z) = \sqrt{x^2 + y^2 + z^2}$$

Argumenti x , y , i z koji se koriste prilikom definicije funkcije nazivaju se **formalni argumenti**. Argumenti koji se koriste prilikom pozivanja funkcije nazivaju se **stvarni argumenti**. U programskom dijelu:

```

a := 3; b := 4; c :=12;
WRITE R(a,b,c)

```

varijable a , b , c , su stvarni argumenti i vrijednost funkcije $R(a,b,c)$ koju ispisuje navedeni program je 13. Program koji poziva neki potprogram naziva se **pozivajući program**, a izračunavanje vrijednosti funkcije naziva se **poziv funkcije**.

Budući potprogrami mogu pozivati jedan drugoga može se smatrati da je pozivani potprogram na nižem hijerarhijskom nivou od pozivajućeg programa ili potprograma. Program na najvišem nivou ove hijerarhije (koji može ali ne mora pozivati potprograme) naziva se **glavni program**.

Osobina funkcija je da imaju jedan ili više ulaznih argumenata i da generiraju jedan skalarni rezultat.

U slučaju potprograma kojim se rješava kvadratna jednačina

$$ax^2 + bx + c = 0$$

imamo tri ulazne veličine (a , b , c) na temelju kojih se izračunavaju dva rezultata: korijeni x_1 i x_2 . Potprogram kojim se obavlja navedena obrada može se označiti na slijedeći način:

KORIJEN (a , b , c , x_1 , x_2),

gdje a , b i c označavaju ulazne, a x_1 i x_2 izlazne veličine. Potprogram ovog tipa naziva se **procedura**.

Na primjer, program

```
a := 1, b := -3; c := 2;  
KORIJEN (a, b, c, x1, x2);  
WRITE x1, x2;  
KORIJEN (1, 0, -1, x1, x2);  
WRITE x1, x2
```

generira i ispisuje najprije rezultate 1 i 2, a zatim 1 i -1.

Samo jedna kopija potprograma smješta se u memoriju i svaki put kada je potrebno da se potprogram izvrši obavlja se skok sa mjesta poziva na mjesto početka potprograma; ovaj skok naziva se **ulazni skok**. Po završetku rada potprograma ostvaruje se **povratni skok** kojim se nastavlja rad u pozivajućem programu neposredno iza mjesta sa kojeg je obavljen ulazni skok.

Osnovni cilj potprograma je omogućiti da se svaka algoritamska cjelina organizira u vidu odgovarajuće programske cjeline. Na taj način postiže se ušteda memorije jer se navedena programska cjelina samo jednom memorira iako se može više puta izvršavati.

Potprogram se definira kao programska cjelina koja u općem slučaju ima slijedeće osobine:

- (1) Potprogram koristi skup ulaznih podataka, obrađuje ih i na temelju obrade formira skup izlaznih rezultata. Jedan od ovih skupova može biti i prazan.
- (2) Potprogram se u okviru nekog pozivajućeg programa može aktivirati ("pozivati") više puta i to svaki put sa drugim skupom ulaznih podataka. Pozivajući program može biti glavni program ili neki od potprograma.
- (3) Potprogram može biti definiran u sastavu pozivajućeg programa. Ta vrsta potprograma naziva se **interni potprogram**. Interni potprogram se ne može nezavisno koristiti izvan programske cjeline unutar koje je definiran.
- (4) Potprogram može također biti definiran kao samostalna cjelina koja se prevodi nezavisno od pozivajućeg programa. Ova vrsta potprograma naziva se **nezavisni potprogram** i on može biti pozvan od bilo kojeg drugog programa ili potprograma.
- (5) Potprogram može direktno ili indirektno pozivati sam sebe i za takav potprogram kaže se da je **rekurzivan**.

4.7 Nezavisne funkcije i njihovi formalni, lokalni, globalni i stvarni argumenti

Nezavisne funkcije (ili funkcijski potprogrami) se u općem slučaju definiraju na način koji opisuje slijedeća sintaksa:

```
FUNCTION naziv_funkcije    (lista_formalnih_argumenata : tip_argumenta;  
                           lista_formalnih_argumenata : tip_argumenta;  
                           .....;  
                           lista_formalnih_argumenata : tip_argumenta);  
                           tip_funkcije;
```

{Definicija lokalnih varijabli, ukoliko postoje}

```
LOCAL lista_lokalnih_varijabli : tip_varijabli;  
      lista_lokalnih_varijabli : tip_varijabli;  
      .....;  
      lista_lokalnih_varijabli : tip_varijabli;
```

{Definicija globalnih argumenata, ukoliko postoje}

```
GLOBAL      lista_globalnih_argumenata : tip_argumenta;  
            lista_globalnih_argumenata : tip_argumenta;  
            .....;  
            lista_globalnih_argumenata : tip_argumenta;
```

{Tijelo funkcije koje ima za cilj izračunati vrijednost funkcije}

```
izvršne instrukcije;  
izvršne instrukcije;  
.....;  
naziv_funkcije := vrijednost;  
.....;  
IF uvjet_završetka THEN RETURN END_IF;  
.....;  
izvršne instrukcije  
  
END_FUNCTION
```

Svaka od navedenih listi može biti prazna, a ako nije prazna može sadržavati proizvoljan broj komponenti.

Svi formalni argumenti funkcije su isključivo **ulazne veličine**. To znači da se vrijednost ovih veličina ne može redefinirati unutar tijela funkcije.

Unutar tijela funkcije koriste se neke lokalne veličine i takve veličine definiramo pomoću specifikacije LOCAL. Lokalne varijable dostupne su isključivo unutar potprograma, a pozivajući program ne može pristupiti ovim veličinama. U većini slučajeva uloga lokalnih varijabli je sadržavati pomoćne veličine, tablične vrijednosti i međurezultate.

Potreba da se dohvate i vrijednosti, koje nisu navedene u listi formalnih argumenata postiže se primjenom specifikacije GLOBAL kojom se definiraju globalni argumenti tj. veličine koje su zajedničke i dohvatljive za sve potprogramme koji sadrže listu globalnih veličina.

Ponekad se uvjeti za izlaz iz potprograma (i za povratni skok) ispune prije nego se dođe do fizičkog kraja funkcijskog potprograma i tada se izlaz iz potprograma ostvaruje naredbom RETURN. Naredba RETURN ima sličnu izlaznu ulogu kao i naredba EXIT, sa razlikom što se RETURN koristi isključivo za funkcijske potprogramme i procedure. Naredba RETURN dovodi do

bezuvjetnog skoka na izlaznu točku potprograma odakle se zatim ostvaruje povratni skok u pozivajući program.

Postoji međutim mogućnost da se i potprogram modificira tako da umjesto preko formalnih argumenata direktno pristupa ulaznim podacima kao globalnim argumentima. U tom slučaju lista formalnih argumenata bila bi prazna i potprogram i pozivajući program imali bi slijedeći oblik:

```
FUNCTION R ( ) : REAL;  
GLOBAL x,y,z : REAL;  
R := sqrt(x*x + y*y + z*z)  
END_FUNCTION
```

```
GLOBAL a, b, c : REAL;  
a := 3; b := 4; c := 12;  
WRITE R ( )
```

Ovdje se u memoriji računala formira **jedinstveno područje globalnih podataka** koje sadrži tri lokacije. Glavni program te lokacije označava se simbolima a,b,c, a potprogram iste lokacije simbolima x,y,z. To znači da su x i a nazivi jedne te iste lokacije, a to jednako važi i za y i b, kao i za z i c. Na taj način ostvaruje se direktan dohvat vrijednosti argumenata pomoću globalnih varijabli.

4.8 Interne funkcije i procedure

Funkcije i procedure ne moraju biti uvijek ostvarene kao nezavisne cjeline. U praksi se često javlja potreba da se funkcije i procedure ostvare kao sastavni dio većih cjelina koji mogu biti neke složenije funkcije i procedure ili glavni program. Osnovna razlika između nezavisnih i internih funkcija i procedura je u mogućnostima pristupa podacima koje definiraju slijedeća tri jednostavna pravila:

- (1) Svi objekti (varijable, tipovi podataka, odredišne oznake skoka, funkcije i procedure) koje definiramo u nekom internom potprogramu predstavljaju lokalne objekte tog potprograma i prema tome nisu dostupni izvan tijela navedenog potprograma.
- (2) Svi objekti koji su definirani u nekom programu ili potprogramu dostupni su u svim internim potprogramima koji su definirani unutar datog (nadređenog) programa ili potprograma.
- (3) U suglasnosti sa prethodnim pravilima, svi objekti definirani u glavnom programu imaju status globalnih objekata i dostupni su svim mjestima unutar cijelog programa i svih internih potprograma.

Da bi se jasno uočila hijerarhija internih funkcija i procedura korisno je eksplicitno naznačiti najviši hijerarhijski nivo glavnog programa. To se postiže tako da se PROGRAM i END_PROGRAM označe kao točke početka i kraja programa i da se programu dodijeli naziv.

```
PROGRAM naziv_programa;
```

```
DEFINE globalne_varijable
```

```
Definicije internih procedura i funkcija
```

```
Izvršni dio (tijelo) glavnog programa
```

END_PROGRAM

4.9 Koprogrami, kaskadna i protočna obrada

Funkcije i procedure obično stvaraju hijerarhijsku strukturu kod koje je pozivajući program na višem nivou, a pozvani potprogram na nižem nivou. Međutim, često se javlja potreba stvaranja niza kooperativnih programskih modula koji se simultano i ravnopravno izvršavaju. Takvi moduli nazivaju se **koprogrami** (engl. coroutines). Koprogrami imaju svojstvo da mogu prekinuti svoje izvršavanje i obaviti prijelaz u neki drugi koprogram primjenom **instrukcije prijenosnog skoka** RESUME.

Opća struktura programa sa koprogramima za slučaj tri koprograma je:

```
PROGRAM GLAVNI;  
DEFINE globalne varijable;  
  
COROUTINE PROG1;  
.....  
RESUME PROG2;  
.....  
RESUME PROG3;  
.....  
RESUME PROG2;  
.....  
END_COROUTINE; {PROG1}  
  
COROUTINE PROG2;  
.....  
RESUME PROG3;  
.....  
RESUME PROG1;  
.....  
RESUME PROG3;  
.....  
END_COROUTINE; {PROG2}  
  
COROUTINE PROG3;  
.....  
RESUME PROG1;  
.....  
RESUME PROG2;  
.....  
RESUME PROG1;  
.....  
END_COROUTINE; {PROG3}  
  
Tijelo glavnog programa  
.....  
PROG1;  
.....  
END_PROGRAM {GLAVNI}
```


Osnovni cilj koji se koprogramima želi postići je omogućiti da međurezultati koje generira jedan program i koje očekuje na daljnju obradu neki drugi program budu dostavljeni na daljnju obradu odmah pošto su generirani u prvom programu. U suprotnom došli bi u situaciju da je potrebno međurezultate spremiti u pomoćne datoteke i zatim organizirati niz nezavisnih obrada (**kaskadna obrada**).

Neki operativni sustavi (npr. UNIX i MS-DOS) omogućavaju jednostavnu organizaciju programskih kaskada; takve kaskade se nazivaju **cijev** (engl. pipe) i povezuju programe (koprograme) u cilju kooperativnog izvršavanja. Alternativni način obrade koji se naziva **protočna obrada** zasniva se na primjeni koprograma. Razlika između navedena dva pristupa je u tome što kaskadna obrada zahtijeva manje memorije za programe (u memoriji je uvijek samo jedan program) ali zahtijeva na disku prostor za datoteke međurezultata. Protočna obrada ne zahtijeva datoteke, ali svi koprogrami (ekvivalentni kaskadnim programima) moraju simultano biti u operativnoj memoriji.